Experiment No. 3

Implementation and Analysis of Spatial Filtering Techniques
using Low-Pass and High-Pass Filters

Aim

To implement and analyze spatial domain filtering techniques using low-pass and high-pass filters for image smoothing, noise reduction, edge enhancement, and feature preservation using Python and OpenCV.

Purpose

Spatial filtering is a fundamental operation in digital image processing that modifies pixel values based on their neighboring pixels. Low-pass filters reduce noise and smooth images, whereas high-pass filters emphasize edges, fine details, and abrupt intensity changes, which are essential for feature extraction and object recognition.

In this experiment, various spatial filters are applied to images and their impact on image quality, edge preservation, and noise removal is evaluated. This shows the significance of filtering as a preprocessing step in medical imaging, surveillance, remote sensing, and autonomous systems.

Prerequisites

- Basic knowledge of digital image processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of image kernels and convolution operations
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a grayscale or color image.
2. Display the original image and analyze its visual characteristics.
3. Apply Gaussian Blur to reduce noise while preserving important structures.
4. Apply Median Filtering to remove impulse (salt-and-pepper) noise.
5. Apply Average (Mean) Filtering and compare it with Gaussian filtering.
6. Implement Laplacian Filtering to enhance edges and fine details.
7. Apply Sobel Edge Detection in the horizontal and vertical directions.
8. Compare low-pass and high-pass filter outputs in terms of noise reduction, edge preservation, and computation time.
9. Display the original and filtered images with observations on strengths and limitations.
10. Interpret the results and discuss applications of spatial filtering.

Key Concepts and Formulas

- **Spatial filtering (convolution/correlation):** g(x, y) = sum over (s, t) of w(s, t) f(x + s, y + t), where w is the kernel.
- **Average filter:** every kernel weight equals 1/(k x k).

- **Gaussian kernel:** $G(x, y) = (1 / 2 \pi \sigma^2) \exp(-(x^2 + y^2) / 2 \sigma^2)$.
- **Median filter:** non-linear; replaces each pixel with the median of its neighborhood.
- **Laplacian:** second-derivative operator, $\nabla^2 f = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$.
- **Sobel:** first-derivative operators G_x and G_y; gradient magnitude = $\sqrt{G_x^2 + G_y^2}$.
- **PSNR** (used in the code to compare filters): $10 \log_{10}(255^2 / \text{MSE})$ dB.

Observations

- The Average filter smooths noise but blurs edges equally in all directions.
- The Gaussian filter smooths with distance-based weights, so it preserves structures better than the average filter and gives higher PSNR for Gaussian noise.
- The Median filter removes salt-and-pepper noise almost completely while preserving edges; Average and Gaussian only spread the impulses into blotches.
- The Laplacian highlights edges in all directions, but is sensitive to noise, so pre-smoothing is used.
- Sobel X responds to vertical edges and Sobel Y to horizontal edges; the magnitude combines both and is more robust to noise than the Laplacian.
- Larger kernels remove more noise but blur edges more; the Median filter is the slowest of the three low-pass filters, and the Average (box) filter is the fastest.

Source Code

File: experiment3_spatial_filtering.py

```
"""
Experiment 3: Spatial Filtering using Low-Pass and High-Pass Filters
Install:  pip install opencv-python numpy matplotlib
Usage:    python experiment3_spatial_filtering.py [path_to_image]
"""
import os
import sys
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt

os.makedirs("output", exist_ok=True)
rng = np.random.default_rng(42)

# ---------------------------------------------------------------
# Step 1 & 2: Load image (grayscale) and display
# ---------------------------------------------------------------
path = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

if img is None:  # fallback synthetic image with sharp edges
    print(f"'{path}' not found - generating a synthetic test image.")
    img = np.full((300, 400), 60, dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (180, 200), 200, -1)
    cv2.circle(img, (290, 130), 70, 150, -1)
    cv2.line(img, (20, 270), (380, 250), 255, 3)
    cv2.putText(img, "Filter", (150, 285), cv2.FONT_HERSHEY_SIMPLEX, 1, 255, 2)

print(f"Image shape: {img.shape}, dtype={img.dtype}")


def psnr(ref, test):
    """Peak Signal-to-Noise Ratio in dB (higher = closer to reference)."""
    mse = np.mean((ref.astype(np.float64) - test.astype(np.float64)) ** 2)
    return float("inf") if mse == 0 else 10 * np.log10(255.0 ** 2 / mse)


# ---------------------------------------------------------------
# Create noisy versions of the image
# ---------------------------------------------------------------
# Gaussian noise
gauss_noise = rng.normal(0, 20, img.shape)
noisy_gauss = np.clip(img + gauss_noise, 0, 255).astype(np.uint8)

# Salt-and-pepper noise (5% of pixels)
noisy_sp = img.copy()
mask = rng.random(img.shape)
noisy_sp[mask < 0.025] = 0       # pepper
noisy_sp[mask > 0.975] = 255     # salt

# ---------------------------------------------------------------
# Steps 3-5: Low-pass filters
# ---------------------------------------------------------------
def timed(fn, *args, repeat=20):
    t0 = time.perf_counter()
    for _ in range(repeat):
        out = fn(*args)
    return out, (time.perf_counter() - t0) / repeat * 1000  # ms

k = 5
gaussian_g, t_gauss = timed(cv2.GaussianBlur, noisy_gauss, (k, k), 0)
average_g, t_avg = timed(cv2.blur, noisy_gauss, (k, k))
median_g, t_med = timed(cv2.medianBlur, noisy_gauss, k)

gaussian_sp = cv2.GaussianBlur(noisy_sp, (k, k), 0)
average_sp = cv2.blur(noisy_sp, (k, k))
median_sp = cv2.medianBlur(noisy_sp, k)

# ---------------------------------------------------------------
# Steps 6-7: High-pass filters (edge detection)
# ---------------------------------------------------------------
# Smooth slightly first - derivative filters amplify noise
smooth = cv2.GaussianBlur(img, (3, 3), 0)

# Laplacian (second derivative): use CV_64F so negative values are kept
lap = cv2.Laplacian(smooth, cv2.CV_64F, ksize=3)
lap_abs = cv2.convertScaleAbs(lap)
# Sobel (first derivative) in x and y
```

```
sobel_x = cv2.Sobel(smooth, cv2.CV_64F, 1, 0, ksize=3)   # vertical edges
sobel_y = cv2.Sobel(smooth, cv2.CV_64F, 0, 1, ksize=3)   # horizontal edges
sobel_mag = cv2.convertScaleAbs(cv2.magnitude(sobel_x, sobel_y))
sobel_x_abs = cv2.convertScaleAbs(sobel_x)
sobel_y_abs = cv2.convertScaleAbs(sobel_y)

# Sharpening: original - Laplacian (edge enhancement)
sharpened = cv2.convertScaleAbs(smooth.astype(np.float64) - lap)

# Manual convolution with a custom kernel (shows the role of kernels)
sharpen_kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]], dtype=np.float32)
custom_sharp = cv2.filter2D(img, -1, sharpen_kernel)

# ---------------------------------------------------------------
# Step 8: Quantitative comparison
# ---------------------------------------------------------------
print("\n---- PSNR vs clean image (dB, higher is better) ----")
print(f"{'Filter':<12}{'Gaussian noise':>16}{'Salt & pepper':>16}")
print(f"{'Noisy':<12}{psnr(img, noisy_gauss):>16.2f}{psnr(img, noisy_sp):>16.2f}")
print(f"{'Average':<12}{psnr(img, average_g):>16.2f}{psnr(img, average_sp):>16.2f}")
print(f"{'Gaussian':<12}{psnr(img, gaussian_g):>16.2f}{psnr(img, gaussian_sp):>16.2f}")
print(f"{'Median':<12}{psnr(img, median_g):>16.2f}{psnr(img, median_sp):>16.2f}")

print(f"\n---- Time per call ({k}x{k} kernel, ms) ----")
print(f"Average : {t_avg:.3f}\nGaussian: {t_gauss:.3f}\nMedian  : {t_med:.3f}")

# ---------------------------------------------------------------
# Step 9: Display results
# ---------------------------------------------------------------
def show(axes, items):
    for ax, (title, im) in zip(axes.ravel(), items):
        ax.imshow(im, cmap="gray", vmin=0, vmax=255)
        ax.set_title(title, fontsize=9)
        ax.axis("off")

fig, axes = plt.subplots(2, 4, figsize=(15, 7))
show(axes, [
    ("Original", img), ("Gaussian noise", noisy_gauss),
    ("Average (Gaussian noise)", average_g), ("Gaussian (Gaussian noise)", gaussian_g),
    ("Salt & pepper noise", noisy_sp), ("Average (S&P)", average_sp),
    ("Gaussian (S&P)", gaussian_sp), ("Median (S&P)", median_sp),
])
plt.tight_layout()
plt.savefig("output/exp3_lowpass.png", dpi=150)
plt.show()

fig, axes = plt.subplots(2, 3, figsize=(12, 7))
show(axes, [
    ("Original", img), ("Laplacian", lap_abs), ("Sobel X (vertical edges)", sobel_x_abs),
    ("Sobel Y (horizontal edges)", sobel_y_abs), ("Sobel magnitude", sobel_mag),
    ("Sharpened (kernel)", custom_sharp),
])
plt.tight_layout()
plt.savefig("output/exp3_highpass.png", dpi=150)
plt.show()

# Effect of kernel size on smoothing (trade-off: noise vs edge blur)
fig, axes = plt.subplots(1, 4, figsize=(15, 4))
show(axes, [(f"Gaussian {s}x{s}", cv2.GaussianBlur(noisy_gauss, (s, s), 0))
            for s in (3, 7, 15, 31)])
plt.tight_layout()
plt.savefig("output/exp3_kernel_size.png", dpi=150)
plt.show()
```

Questions and Answers

1. What is spatial filtering? How is it used in digital image processing?

Spatial filtering modifies each pixel's value using the values of its neighbors in the image plane, by sliding a small mask (kernel) over the image and computing a weighted sum (linear) or a rank/statistic (non-linear) of the neighborhood. It is used for:

- Smoothing and noise reduction (low-pass)
- Sharpening and edge detection (high-pass)
- Preprocessing for segmentation, feature extraction, and recognition
- Image enhancement and restoration

2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Aspect	Low-Pass Filter	High-Pass Filter
Passes	Slow intensity variations (smooth regions)	Rapid intensity changes (edges, fine detail, noise)
Effect	Blurs and smooths, reduces noise	Sharpens, highlights edges
Kernel	Positive weights summing to 1	Positive and negative weights summing to 0 (for edge filters)
Examples	Average, Gaussian, Median	Laplacian, Sobel, Prewitt, unsharp masking
Drawback	Loses edges and detail	Amplifies noise

3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Aspect	Average	Gaussian	Median
Principle	Linear; equal-weight mean of neighborhood	Linear; weighted mean, center pixels weigh more (bell curve)	Non-linear; median of sorted neighborhood values
Edge preservation	Poor; blurs edges	Moderate; better than average	Good
Best against	Mild random noise	Gaussian noise	Salt-and-pepper (impulse) noise
Speed	Fastest	Fast (separable)	Slowest (requires sorting)
Applications	Quick smoothing, background estimation	Pre-smoothing before edge detection (Canny), downsampling	Scanned documents, sensor/transmission errors, medical images

4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Salt-and-pepper noise consists of isolated pixels at extreme values (0 or 255). These outliers lie at the ends of the sorted neighborhood, so the median ignores them and picks a typical neighborhood value. The impulse is removed completely rather than averaged into surrounding pixels (as the mean filter does, leaving blotches), and the output is always an actual pixel value from the neighborhood, so edges remain sharp and no new gray levels are created.

5. Explain the role of convolution kernels in spatial filtering.

A kernel (mask) is a small matrix of weights (e.g., 3 x 3 or 5 x 5) that is slid over the image. At each position, the kernel is multiplied element-wise with the underlying pixels and the products are summed to give the output pixel. The kernel's values define the filter's behavior:

- All positive weights summing to 1 give smoothing (average, Gaussian)
- Mixed positive and negative weights summing to 0 give edge detection (Sobel, Laplacian)
- Kernel size controls the strength and scale of the effect

- In OpenCV, any custom kernel can be applied with `cv2.filter2D`

6. What is the purpose of the Sobel and Laplacian operators in edge detection?

- **Sobel:** a first-derivative operator that estimates the intensity gradient along x and y (with built-in Gaussian-like smoothing). It gives edge strength and direction, and is relatively robust to noise.
- **Laplacian:** a second-derivative operator that responds to rapid intensity change in all directions (isotropic). Edges appear at zero crossings. It highlights fine details and is used for sharpening, but is highly noise-sensitive and gives no direction information.

7. Why are filtering operations considered an essential preprocessing step in computer vision?

Real images contain noise from sensors, low light, and transmission, which produce false edges and unstable features. Filtering removes this noise, improves edge and feature detection accuracy, normalizes image quality, and reduces variability so that algorithms like segmentation, feature matching, optical flow, and CNN-based detection work more reliably. Sharpening filters also emphasize details needed for recognition.

8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Noise and edges are both high-frequency content, so a filter that suppresses noise also tends to suppress edges. Larger kernels or higher sigma give stronger noise reduction but blur boundaries and fine details. Small kernels preserve edges but leave more noise. Linear filters (average, Gaussian) blur edges, while edge-preserving filters (median, bilateral, non-local means) trade extra computation for better edge retention. The kernel size and filter type must be chosen to suit the noise type and the detail that matters for the task.

9. Mention four real-world applications where spatial filtering techniques are widely used.

1. Medical imaging: noise reduction and edge enhancement in X-ray, CT, and MRI
2. Surveillance and license plate recognition: denoising and sharpening
3. Remote sensing and satellite imagery: smoothing, feature and boundary extraction
4. Autonomous vehicles and robotics: lane and obstacle edge detection
5. (Also: OCR and document scanning, industrial defect inspection)

10. Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Aspect	Spatial Domain	Frequency Domain
Operates on	Pixels directly, using a kernel (convolution)	Fourier transform of the image, using a filter H(u, v) multiplied with F(u, v)
Implementation	Simple, intuitive, supports non-linear filters (median)	Requires FFT, filtering, and inverse FFT; linear only
Cost	Efficient for small kernels	More efficient for large kernels (convolution becomes multiplication)
Design	Hard to design precise frequency response	Easy to design exact cut-off (ideal, Butterworth, Gaussian filters)
Applications	Noise removal, sharpening, edge detection in real-time systems	Periodic noise removal, notch filtering, image restoration, compression

Conclusion

Low-pass filters (Average, Gaussian, Median) and high-pass filters (Laplacian, Sobel) were implemented using Python and OpenCV and compared through images, PSNR, and execution time. Gaussian filtering is best for general smoothing of Gaussian noise, the Median filter is best for impulse noise, and Sobel gives robust edge detection while the Laplacian emphasizes fine detail but needs pre-smoothing. The right

filter depends on the noise type and the edge detail that must be preserved.